Java Fundamentals Assignment 1

Submitted By :-
Naman Patel

[Note :- This PDF contains theory questions, while programming questions are uploaded on github **]**

Data Types and Operators :-

1) **Explain the difference between primitive and reference data types with examples.**

Primitive Data type :-

- These are the basic data types that store actual values directly in memory.
- They are used to store single values.
- They do not have any methods.
- Examples :- int, char, double, boolean

```
int age = 20;
double price = 99.5;
char grade = 'A';
```

Reference Data type :-

- These are the data types that do not store actual values, instead they store address of an object in memory.
- They are used to store multiple values.
- They can have methods.
- Examples :- String, arrays, objects

```
String name = "Naman";
int[] arr = {1, 2, 3};
Student s = new Student();
```

Demo Program about working of Primitive and Reference datatype

```
public class PrimitiveVsReference {

   // Primitive datatype
   public static void Primitive() {
      int a = 10;
      int b = a                          // value of 'a' copied
      b = 20;
    System.out.println(a);       //10  (here value of 'a' does not change by changing value of 'b')
   }

   // Referenec datatype
   public static void Reference() {
      int[] arr1 = {1, 2, 3};
      int[] arr2 = arr1;                      // reference of arr1 copied
      arr2[0] = 99;
      System.out.println(arr1[0]);      //99  (here value of arr1 gets changed by changing value of arr2)
   }

  public static void main(String[] args) {
      // calling both methods
      Primitive();
      Reference();
   }
}
```

Object Oriented Programming (OOP):

1) **Explain the concept of encapsulation with a suitable example.**

- Encapsulation means binding of variables and methods into a single unit (i.e. class) and allowing only limited access to data through access specifiers.
- Access specifiers like public, private, protected and default are used to control access.
- It helps to protect data from unwanted access and makes code secure and organized
- Example :- In student class we can keep marks as private and it can only be accessed through setter and getter method.

```
class Student {
    private int marks;                          // here marks cannot access directly
    public void setMarks(int marks) {       // method to set new value in marks
      this.marks = marks;
    }
    public int getMarks() {                     // method to get marks
      return marks;
    }
}

class Main {
   public static void main(String[] args) {
      Student s = new Student();
      s.setMarks(80);
      System.out.println(s.getMarks());
   }
}
```

Advanced Topics :

1) **Explain Explain the concept of interfaces and abstract classes with examples.**

Interface :-

- Interface contains only abstract method (no body) by default
- Variables are public + static + final
- It is used to achieve 100% abstraction.
- It require subclass to provide implementation and a class can implement multiple interfaces and hence supports multiple inheritence
- Example :-

```
interface Animal {
    void sound();           // abstract method
}


class Dog implements Animal {
    public void sound() {                   // provide implementation to sound() method
    System.out.println("Dog barks");
    }
}

public class Test {
 public static void main(String[] args) {
    Puppy p = new Puppy();
    p.sound();
   }
}
```

Abstract Class:-

- Abstract class can have both abstract method (no body) and normal method (with body)
- It is used when we want partial abstraction.
- A class can extend only one abstract class.
- Example :-

```
abstract class Dog {
   abstract void eat();          // abstract method

   void sleep() {                // normal method
      System.out.println("Dog is sleeping");
   }
}

class Puppy extends Animal {
   void eat() {                  // provide implementation to eat() method
      System.out.println("Puppy eats food");
   }
}

public class Test {
   public static void main(String[] args) {
      Puppy p = new Puppy();
      p.eat();
      p.sleep();
   }
}
```

2) Explore multithreading in Java to perform multiple tasks concurrently.

- Multithreading allows multiple tasks (or threads) to run at the same time.
- It Improves performance and efficiency
- Threads can be created in 2 ways :-
  i. By Extending Thread class
  ii. By Implementing Runnable
- Example :- Listening to music + downloading file at same time.

<u>Multithreading program :-</u>

```
class MyThread extends Thread {

    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println("Thread 1 :- " + i);
        }
    }
}

public class MultiThreading {
    public static void main(String[] args) {

        MyThread t1 = new MyThread();       //creating mythread object
        t1.start();                         // starts new thread

        // this loops run and main thread also runs independently
        for (int i = 1; i <= 5; i++) {
            System.out.println("Main Thread :- " + i);
        }
    }
}
```